Universidad de Oviedo
Universidá d'Uviéu
University of Oviedo

Escuela de Ingeniería Informática
Universidad de Oviedo

DEGREE IN SOFTWARE ENGINEERING

Type of TFG: Development

Year: 2025 / 2026

Population growth simulator. Simulation of population models with stochasticity

Author: Mario Orviz Viesca

Tutors: Celia Melendi Lavandera

Mario Quevedo de Anta

Version 0.1 (25/02/2026) Oviedo, Spain

Important Notes on this Template

VERY IMPORTANT NOTE: *This template is only an* ***orientative guide****. A Final Degree Project (TFG) does not necessarily have to include all these sections, nor only those that appear here; it depends on the specific characteristics of each project.*

USING THIS TEMPLATE IS NOT MANDATORY AT ALL *for completing a TFG; it is only an optional aid if the tutor or student wishes to use it.*

The template must be adapted to each project (adding sections, removing some, or modifying the contents as needed for each particular case). The student must always consult their director, whose instructions ***ALWAYS*** *take priority over any doubts regarding any topic.*

This template has been prepared thanks to the help of Pablo Javier Tuya González, professor in charge of the subjects Software Process Engineering *and* Software Quality, Validation, and Verification*, as well as other professors such as Raúl Izquierdo, Aquilino Fuente, Benjamín López, Jose Emilio Labra, etc., who provided feedback on the parts related to their subjects. Your decisions must be consistent with what is taught in these subjects and others applicable to each part of the developed project (e.g.,* Project Management and Planning*,* Computer System Security*,* Databases*,* Programming Language Design*, etc.). Some of these are explicitly mentioned in various sections of this document.*

Although this document is provided as a guide, nothing prevents using it only as a point-of-reference guide and generating the documentation automatically in various formats following the ***"Documentation as Code"*** *philosophy learned during the degree. You do not have to use Word; other formats like* ***Typst****,* ***LaTeX****, etc., are perfectly possible.*

WE REITERATE THAT THE USE OF THIS DOCUMENT IS NOT MANDATORY*, and the content of* ***RELATED SUBJECTS WILL ALWAYS PREVAIL*** *over what is stated here. The idea of this document is to serve as an* ***AID****, NOT as a* ***RULE****.*

This document may contain errors. If you wish to collaborate to improve it, please send an email with your issues to `redondojose` *at the Uniovi email domain. THANK YOU.*

Aspects to consider for your TFG documentation:

- *The explanatory text for the template parts is in brown and* ***MUST DISAPPEAR COMPLETELY*** *from the final version. For your own text, use black color. Leaving template text in the TFG documentation will have a significant negative impact on your grade, as it indicates a lack of care in your work.*

- ***Images and Tables:*** *To ensure they are included in the indices, use the* `#figure(...)` *function. It is recommended to centre them and ensure they are legible and of high quality. To respect copyrights,* ***always cite the source (URL)*** *from which they were obtained. You can add it at the end of the caption (e.g., "Source: <URL>").*
- ***Hyperlinks and Navigation:*** *When generating the final PDF, ensure that bookmarks are generated so the resulting document is navigable in a PDF viewer, facilitating the tribunal's work.*
- ***Digital Quality:*** *Generate the PDF directly (Export/Save as) instead of "Printing to PDF" to ensure that internal document links and external URLs remain clickable.*
- ***Spelling and Grammar:*** *Always run the spell checker at the end and check for grammatical errors. Failing to do so diminishes the quality of your work.*
- ***Research Component:*** *If your project has a strong research character, there is a specific template for research projects for the Master's in Web Engineering. It can be combined with this one if your development project includes a research component.*

Figure 1: Example of a figure with a citation. Use "Figure" or "Illustration" consistently throughout the document. Source: www.uniovi.es

Declaration of Authorship and Originality

In accordance with the provisions of Article 8.3 of the Agreement of July 17, 2020, of the Governing Council of the University of Oviedo, approving the Regulations on the preparation and defense of master's degree final projects at the University of Oviedo (BOPA No. 153 of 7-viii-2020):

Mr./Ms. **(Student Name and Surname)**, with ID number **(DNI number)**,

I DECLARE THAT:

The Final Degree Project titled **Population growth simulator. Simulation of population models with stochasticity**, which I present for exhibition and defense, is original and I have duly cited all sources of information used, both in the body of the text and in the bibliography.

_____

«Digital Signature of the Student»

In Oviedo, on

_____

of

_____

of 20

___

ACKNOWLEDGEMENTS

This section is by no means mandatory, but it is the correct place to dedicate the project to the people, institutions, or companies you wish.

I would like to thank…

TABLE OF CONTENTS

List of Figures

List of Tables

Chapter 1

General Description of the Project

1.1 Summary

This project is a web application developed in collaboration with the Faculty of Biology, designed to simulate population growth dynamics for educational purposes. Its main goal is to serve as an interactive teaching tool that helps university students and researchers or professors better understand and visualize how populations evolve over time.

The application is entirely web-based, meaning it requires no installation and can be accessed from any device with a browser and an internet connection. It is intended to be free and openly available to anyone in the world, with no barriers to access. This global accessibility is a core design principle of the project, as the aim is to bring quality educational resources to as wide an audience as possible, regardless of their institution or geographic location.

The project was born out of a real collaboration between the School of Engineering and the Faculty of Biology, which gives it a genuine interdisciplinary and practical context. Although the application is currently functional and actively under development, the intention is for it to eventually be deployed and used in real academic settings, both as a complement to biology courses and as a resource for independent study and research.

By combining an intuitive interface with solid scientific foundations, the application bridges the gap between theoretical population ecology and hands-on learning, making abstract mathematical models tangible and interactive for its users.

1.2 Keywords

Population dynamics, Educational simulation, Web application, Computational biology, Interactive visualization

1.3 Abstract

This project presents the design and development of a web application created in collaboration with the Faculty of Biology, aimed at simulating population growth dynamics for educational purposes. The tool allows users to interactively explore classical population models, such as exponential and logistic growth, through intuitive visual representations that make abstract mathematical concepts more accessible and engaging.

The application is entirely browser-based, requiring no installation, and is freely available to anyone with an internet connection. It targets university students, researchers, and professors as its primary audience, serving as a complementary resource for teaching and studying population ecology.

The project emerges from a genuine interdisciplinary collaboration between the fields of software engineering and biology, with the long-term goal of deploying the tool in real academic environments worldwide. The current version is functional and under active development, with a focus on usability, scientific accuracy, and universal accessibility.

Chapter 2

Planning and Management

This section is COMPULSORY. You must follow the guidelines provided by the 4th-year subject "Project Management and Planning" to develop a plan that details the project activities, participants, timelines, and responsibilities for each, the expected results, and the work plan to be followed for your TFG.

When preparing the planning and budget, keep in mind that there will be several tasks parallel to the project's creation: Documentation, Testing, Project Management... these must be reflected in the planning to achieve a better representation of the work actually performed. Also, note that the different roles the student will occupy (analyst, developer...) do not have the same cost in the budget. You should consider assigning a different cost according to the role occupied by the student in each planned phase.

For the planning, a Gantt chart must be included that allows all high-level tasks, their duration, and interrelationships to be seen on a single page. If you wish to show more detail, partial diagrams can be made.

In the case of using agile methodologies, the Gantt chart may show the different iterations (sprints and/or releases) and other additional activities (e.g., training, previous studies, documentation...). It can be accompanied by a summary story mapping to show more detail. If time tracking has been performed, include the Burn-Down Chart.

This is the advisable content structure, but it is recommended that, at a minimum, there is an initial plan, a risk list, an initial budget, and a final plan and budget.

2.1 Project Planning

2.1.1 Identification of Stakeholders

Since this was already detailed in a previous section, reference it here.

The following table shows, identified by a unique ID and classified into external and internal, the different stakeholders identified for the system "Population growth simulator. Simulation of population models with stochasticity"

Stakeholders		
ID	**Name**	**Description**
Internal Stakeholders		
SI-1	Universidad de Oviedo	University to whom the TFG belongs
SI-2	Biology Faculty of the University of Oviedo	Faculty for whom the software is developed
SI-3	Software Engineering School of the University of Oviedo	School to whom the student that will develop the software belongs
SI-4	Celia Melendi Lavandera	TFG tutor belonging to the Software Engineering School
SI-5	Mario Quevedo de Anta	TFG co-tutor belonging to the Biology Faculty
SI-6	Mario Orviz Viesca	Student in charge of the planning, development, and documentation of this TFG
SI-7	Teachers and students from the Biology Faculty	Actual real users for the application
External Stakeholders		
SE-1	Max Planck Institute for Demographic Research (Germany)	Institution that supports and hosts the COM(P)ADRE online repository for matrix population models (MPMs) and metadata
SE-2	Other biology organisms	Some other biology organisms that might find this tool useful for their activity
SE-3	Hosting provider	This product will be a web product, it will need a hosting provider

Table 1: Internal and External Stakeholder identification

2.1.1.1 System users

The following table shows the different types of users that are expected to interact with the future system:

System Users		
ID	**Name**	**Description**
SU-1	Non-registered user	User that accesses the system without having an account or being logged in
SU-2	Registered user	User that accesses the system being logged into their account

Table 2: System Users

2.1.2 OBS and PBS

In this section, I will present the organizational breakdown structure and the product breakdown structure for this project.

2.1.2.1 OBS

This section defines the **Organizational Breakdown Structure (OBS)** for a project in which one person performs all tasks, supported by two tutors who provide guidance, feedback, and formal acceptance.

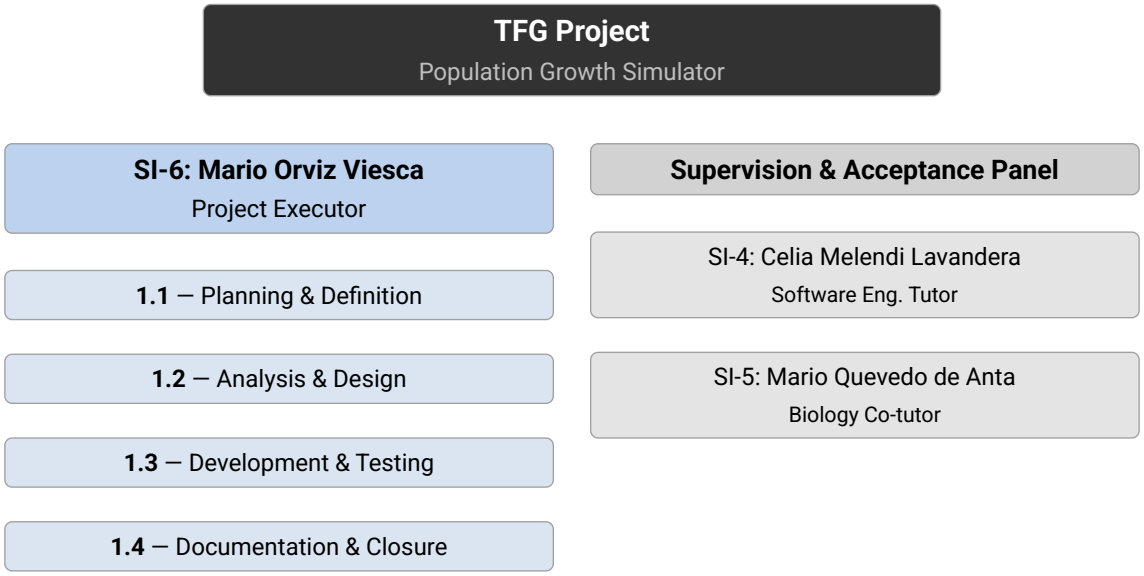

Figure 2: Organizational Breakdown Structure (OBS)

OBS Nodes		
ID	**Node**	**Description**
1. Project Executor (SI-6)		
1.1	Planning & Definition	Scope definition, scheduling, stakeholder analysis, and risk planning.
1.2	Analysis & Design	Requirements elicitation, system architecture, and interface design.
1.3	Development & Testing	Implementation of all system components and verification activities.
1.4	Documentation & Closure	TFG report writing, final delivery, and project closure tasks.
2. Supervision & Acceptance Panel		
2.1	SI-4 — Celia Melendi Lavandera	Software Engineering tutor. Provides methodological guidance and reviews technical deliverables.
2.2	SI-5 — Mario Quevedo de Anta	Biology co-tutor. Validates domain correctness and grants formal acceptance of the biological models implemented.

Table 3: OBS node descriptions

Responsibility Assignment Matrix (RACI)			
Activity	**SI-6 (Executor)**	**SI-4 (Tutor)**	**SI-5 (Co-tutor)**
Planning & Definition	R	C	I
Requirements & Analysis	R	C	A
Architecture & Design	R	C	I
Development	R	I	I
Testing & Validation	R	C	A
TFG Documentation	R	A	I
Mid-project Review	R	C	A
Final Delivery & Closure	R	A	C

Table 4: RACI matrix — **R**: Responsible, **A**: Accountable/Accepts, **C**: Consulted, **I**: Informed

2.1.3 Initial Planning. WBS

Use a Gantt chart that is legible (the text shown in the graphic must be readable), if necessary divided into parts and using task nesting levels, explained and analyzed (present tables with information about the activities).

2.1.4 Risks

2.1.4.1 Risk Management Plan

(Referenced here. Included in the Appendix)

2.1.4.2 Risk Identification

At least 10 risks must be identified.

2.1.4.3 Risk Register

The risk register should be included here; if risk sheets are developed, they should be placed in an Appendix.

2.1.5 Initial Budget

2.1.5.1 Cost Budget

Detailed estimation of the internal costs of the project.

2.1.5.2 Client Budget

This section will be included when required.

2.2 Project Execution

2.2.1 Planning Tracking Plan

Explanation of the tracking plan. At least the description of 3 baselines should be introduced (initial, mid-project, and final).

2.2.2 Project Issue Log

Ideally, relate it to the planning update.

2.2.3 Risks

Follow-up of at least 5 risks. Include the risk sheets. Ideally, relate the events in the issue log to the occurrence of risks if they appear.

2.3 Project Closure

2.3.1 Final Planning

2.3.2 Final Risk Report

2.3.3 Final Cost Budget

2.3.4 Lessons Learned

Chapter 3

STAKEHOLDER REQUIREMENTS

The purpose is to provide an initial definition of the requirements for a system that can provide the services needed by users and other stakeholders. This involves a preliminary description of what the system to be developed should include.

As a guideline, the content of this chapter corresponds to the results of the "Stakeholder Requirements Definition" process in ISO/IEC 15288 or the "System Feasibility Study (EVS)" process in the Métrica Version 3 methodology.

3.1 SYSTEM SCOPE

Identify the users and other stakeholders in the system and briefly describe their objectives and needs. It may also include a description of the system's operation and/or processes as they are currently performed.

3.2 Stakeholder Requirements

This will include the identified requirements, usually organized in a structured way using a hierarchical list. It will address both functional and non-functional requirements and other constraints that condition the final product. Note that these requirements should be written from the user's point of view and must be brief and concise. As a guideline, they may occupy two or three pages.

3.2.1 Functional Requirements

SR-F-01. Authentication: The system shall provide mechanisms for users to register, access, and leave the system securely.

- **SR-F-01.1. User Registration:** The system shall allow new users to create an account.
  - **SR-F-01.1.1. Registration Fields:** The system shall require a username, an email address, and a password to create an account.
  - **SR-F-01.1.2. Username Uniqueness:** The system shall reject a registration attempt if the provided username is already associated with an existing account.
  - **SR-F-01.1.3. Email Uniqueness:** The system shall reject a registration attempt if the provided email address is already associated with an existing account.
  - **SR-F-01.1.4. Password Policy:** The system shall require a password of at least 8 characters to create an account.
  - **SR-F-01.1.5. Secure Password Storage:** The system shall store user passwords in a secure, non-recoverable form. Plain-text passwords shall never be persisted.
- **SR-F-01.2. User Login:** The system shall allow registered users to authenticate and obtain access to protected resources.
  - **SR-F-01.2.1. Login Credentials:** The system shall allow users to log in using either their username or their email address, combined with their password.
  - **SR-F-01.2.2. Credential Validation:** The system shall reject login attempts where the provided credentials do not match any registered account.
  - **SR-F-01.2.3. Session Credential:** Upon successful login, the system shall issue a session credential to the client for use in subsequent authenticated requests.
  - **SR-F-01.2.4. Failed Login Feedback:** The system shall return a generic error message on failed login attempts without disclosing which credential was incorrect.
- **SR-F-01.3. Session Management:** The system shall maintain authenticated user sessions across page interactions.

SR-F-01.3.1. Session Persistence: The system shall preserve the user's authenticated session across page reloads without requiring re-authentication.

SR-F-01.3.2. Session Expiry: The system shall automatically expire sessions after a defined period of inactivity.

SR-F-01.3.3. Invalid Session Rejection: The system shall deny access to protected resources when the session credential is absent, invalid, or expired.

SR-F-01.4. User Logout: The system shall allow authenticated users to terminate their session.

SR-F-01.4.1. Session Termination: Upon logout, the system shall revoke the user's active session so that it cannot be reused to access protected resources.

SR-F-02. Population Matrix Catalog: The system shall provide a publicly accessible catalog of population projection matrices sourced from the COMPADRE Plant Matrix Database.

SR-F-02.1. Matrix Browsing: The system shall display a list of available population projection matrices to any visitor without requiring authentication.

SR-F-02.2. Matrix Search and Filtering: The system shall allow visitors to search and filter the matrix catalog by relevant attributes such as species name or taxonomic group.

SR-F-02.3. Matrix Detail View: The system shall display the full details of a selected matrix, including its metadata and numerical values.

SR-F-02.3.1. Matrix Life Cycle Diagram: The system shall display a life cycle diagram alongside the matrix detail view, representing life-history stages as nodes and transition values as directed, labelled edges.

SR-F-03. Matrix Management: The system shall allow authenticated users to create, import, export, and manage their own population projection matrices.

SR-F-03.1. Matrix Creation: The system shall allow an authenticated user to define a new population projection matrix by entering its numerical values and descriptive metadata.

SR-F-03.2. Matrix Import: The system shall allow an authenticated user to load a population projection matrix from a local file.

SR-F-03.3. Matrix Export: The system shall allow an authenticated user to download any matrix accessible to them as a local file.

SR-F-03.4. Matrix Persistence: The system shall allow an authenticated user to save a matrix to their account so that it is stored in the application and accessible across sessions.

SR-F-03.5. Matrix Editing: The system shall allow an authenticated user to modify the values and metadata of matrices they own.

SR-F-03.6. Matrix Visibility Control: The system shall allow an authenticated user to set the visibility of any matrix they own.

SR-F-03.6.1. Public Visibility: A matrix set to public shall be visible and accessible to all users of the application, including unauthenticated visitors.

SR-F-03.6.2. Private Visibility: A matrix set to private shall be visible and accessible only to its owner.

SR-F-03.6.3. Shared Visibility: A matrix set to shared shall be visible and accessible only to its owner and to the specific users with whom it has been shared.

SR-F-03.7. Matrix Sharing: The system shall allow an authenticated user to share any matrix they own with one or more specific registered users.

SR-F-04. Population Simulation: The system shall allow authenticated users to create, configure, run, manage, and export population growth simulations.

SR-F-04.1. Simulation Creation: The system shall allow an authenticated user to create a new simulation by selecting one or more projection matrices, specifying an initial population vector, and choosing the number of time steps to run.

SR-F-04.1.1. Deterministic Simulation: The system shall allow the user to run a deterministic simulation using a single projection matrix, computing the population vector at each time step by multiplying the current vector by the selected matrix.

SR-F-04.1.2. Stochastic Simulation: The system shall allow the user to run a stochastic simulation by selecting two or more projection matrices of matching dimensions and species; at each time step the system shall select one of the matrices at random to advance the population vector.

SR-F-04.2. Matrix Selection for Simulation: The system shall allow the user to select multiple matrices for a stochastic simulation and shall only permit matrices of the same dimensions to be combined in a single simulation run.

SR-F-04.3. Results Visualization: The system shall display the results of a completed simulation as a chart showing the population size per stage across all simulated time steps.

SR-F-04.4. Demographic Analysis: The system shall compute and display key demographic metrics derived from the projection matrix used in a deterministic simulation, or from a mean matrix when working in a stochastic context.

SR-F-04.4.1. Dominant Growth Rate: The system shall compute and display the dominant eigenvalue (λ) of the projection matrix, representing the asymptotic per-capita population growth rate.

SR-F-04.4.2. Stable Stage Distribution: The system shall compute and display the stable stage distribution, derived from the right eigenvector associated with the dominant eigenvalue, representing the proportional composition of each life-history stage at demographic equilibrium.

SR-F-04.4.3. Reproductive Values: The system shall compute and display the reproductive value for each stage, derived from the left eigenvector associated with the dominant eigenvalue, representing the relative contribution of each stage to future population growth.

SR-F-04.4.4. Sensitivity Analysis: The system shall compute and display the sensitivity matrix, quantifying how an absolute change in each matrix element affects the dominant eigenvalue.

SR-F-04.4.5. Elasticity Analysis: The system shall compute and display the elasticity matrix, quantifying the proportional contribution of each matrix element to the dominant eigenvalue.

SR-F-04.5. Stochastic Analysis: The system shall compute and display additional demographic statistics when the user runs a stochastic simulation.

SR-F-04.5.1. Stochastic Population Summary: The system shall display summary statistics of the stage population sizes at the final time step across all replicate simulation runs, including the median and confidence intervals for each stage.

SR-F-04.5.2. Stochastic Growth Rate: The system shall estimate and display the stochastic population growth rate (log λ_s), reporting the theoretical approximation (Tuljapurkar's method), the simulated rate, and confidence intervals for the simulated rate.

SR-F-04.5.3. Quasi-Extinction Probability: The system shall compute and display a quasi-extinction probability curve showing the cumulative probability that the total population size falls below a user-defined threshold at or before each simulated time step, across all replicate runs.

SR-F-04.6. Simulation Persistence: The system shall allow an authenticated user to save a simulation to their account so that its parameters and results are stored in the application.

SR-F-04.7. Simulation Modification: The system shall allow an authenticated user to modify the parameters of a saved simulation and re-run it.

SR-F-04.8. Simulation Export: The system shall allow an authenticated user to download the parameters and results of any of their simulations as a local file.

SR-F-04.9. Simulation Import: The system shall allow an authenticated user to load a previously exported simulation from a local file and restore its parameters and results.

SR-F-04.10. Simulation History: The system shall maintain a history of the authenticated user's saved simulations and allow them to review and manage past runs.

SR-F-04.10.1. History List: The system shall present the authenticated user with a list of their saved simulations.

SR-F-04.10.2. History Detail: The system shall allow the user to view the full parameters and results of any saved simulation.

SR-F-04.10.3. Simulation Deletion: The system shall allow the authenticated user to permanently delete any of their own saved simulations.

3.2.2 Non-Functional Requirements

SR-NF-01. Performance: The system shall respond to any user interaction within 3 seconds under normal load conditions (100 concurrent users).

SR-NF-02. Availability: The system shall be available at least 99.5% of the time, measured on a monthly basis, excluding scheduled maintenance windows.

SR-NF-03. Security: The system shall restrict access to authenticated-only functionalities to registered and logged-in users.

SR-NF-04. Maintainability: The system shall be designed in a modular way so that any individual component can be updated or replaced with minimal impact on the rest of the system.

SR-NF-05. Usability: A new user with basic computer skills shall be able to complete the main tasks without prior training in less than 5 minutes.

SR-NF-06. Portability: The system shall operate correctly on the following web browsers: Chrome, Firefox, Opera, Safari. No client-side installation shall be required beyond a standard web browser.

SR-NF-07. Legal & Regulatory Compliance: The system shall comply with applicable regulations, including GDPR / LOPD-GDD for personal data protection, the LPI and the LSSICE.

3.2.3 System constraints

3.3 Alternatives

When there is freedom to choose between several alternatives (both functional and technical) to comply with the user requirements, a description of their pros and cons and the justification for the selected alternative should be included.

This section is divided into two subsections: "Solution Alternatives" and "Technology Alternatives". The former describes the different solutions presented for this project (web app, desktop app (with Python or with a `.exe` file), a python script or Jupyter notebook…).

The latter describes the different technical alternatives considered to build the project, the selected programming language (Python) for the frontend (used Python Shiny as a constraint from the stakeholder), backend (FastAPI), the database selection (relational Postgres database), the migration technology, and the ORM used.

3.3.1 Solution Alternatives

Before committing to a specific delivery format, four plausible approaches were evaluated: distributing a plain Python script or Jupyter notebook, packaging the application as a platform-native compiled executable, running the tool as a local desktop application with a graphical interface, and hosting it as a web application accessible through a browser. Each alternative was assessed against the project's primary goals: universal accessibility, integration with the COMPADRE database, multi-user collaboration, and zero installation friction.

3.3.1.1 Plain Python Script / Jupyter Notebook

The simplest possible delivery format would be a Python script (`.py`) or an interactive Jupyter notebook (`.ipynb`) that users download and run locally. The simulation mathematics — matrix multiplication with NumPy [1] — are a natural fit for this environment, and the barrier to *development* is minimal.

However, this approach places the entire setup burden on the end user. Every person wanting to run a simulation must first install Python at the correct version, create a virtual environment, resolve dependency conflicts between NumPy, SciPy [2], and any visualisation library, and keep everything up to date. In an academic context where target users are biology students and researchers — not software engineers — this friction is a significant deterrent. There is also no graphical interface: the user interacts through a terminal or a notebook cell, which is unsuitable for an audience unfamiliar with command-line tools.

Beyond usability, the model offers no persistence or sharing. Each user maintains their own disconnected copy of any saved matrices or simulation results. Integrating the COMPADRE Plant Matrix Database [3] (6,000+ matrices) would require either bundling a large static file with every distribution or asking users to fetch and manage it themselves — both unacceptable for a tool meant to serve a broad, non-technical audience. Dependency conflicts (Python version mismatches, `pip` environment pollution) are a recurring

source of failure in academic computing environments, making this option fragile in practice.

3.3.1.2 Standalone Desktop Application (.exe)

Packaging the application as a compiled, platform-native executable using tools such as PyInstaller [4] or cx_Freeze removes the requirement for users to have Python installed. The runtime is bundled into the binary, and the user's experience resembles installing any conventional desktop software.

The appeal here is real: a single double-click gets the tool running, with no environment setup and no command line. However, the practical drawbacks are substantial. Because the Python interpreter is bundled, the resulting executable is large — typically 150–300 MB for a moderately complex application. More critically, compiled executables are platform-specific: separate build artefacts must be produced and maintained for Windows, macOS, and Linux, each requiring its own signing and distribution pipeline.

Updates present a further problem. There is no mechanism to push a fix or a new COMPADRE snapshot to installed copies; every user must manually download and reinstall the application. University and corporate IT environments frequently block or quarantine unsigned executables, making deployment to the most common user environment — a university workstation — unreliable. Finally, every installed instance is isolated: there is no shared database, no user accounts, and no way to share a custom matrix between two colleagues running the tool on different machines.

3.3.1.3 Local Desktop Application (Python)

A richer alternative to a raw script is a locally-hosted graphical application built with a Python GUI framework such as Python Shiny [5] (running on localhost), tkinter, or PyQt6. This preserves the use of the Python ecosystem while providing a reactive, point-and-click interface close in feel to the final product.

Python Shiny in particular is attractive because it shares the same programming model as the chosen solution and could, in principle, reuse large portions of the application code. The tool would work offline once installed, and the simulation logic would remain entirely on the user's machine.

In practice, however, this approach still requires the user to install Python, Shiny, NumPy, matplotlib, and all transitive dependencies — precisely the environment setup problem identified above. Python Shiny's local mode launches a local HTTP server and opens a browser tab, but it is not a true desktop application: it depends on available ports, can be disrupted by firewalls, and produces an experience indistinguishable from a web app — without any of the distribution benefits of one. Each user's data remains local, collaborative features are impossible, and the live COMPADRE database still requires a network connection regardless of the "offline" framing. In effect, this alternative inherits all the drawbacks of the script approach while adding the complexity of a GUI framework.

3.3.1.4 Web Application

The chosen solution is a three-tier web application: a Python Shiny frontend served at port 8888, a FastAPI REST API at port 8000, and a PostgreSQL database — all orchestrated with Docker Compose [6] and accessible from any modern browser.

This architecture directly satisfies every constraint the other alternatives failed to meet. The user requires nothing beyond Chrome or Firefox; there is no installation, no version management, and no environment to maintain. The COMPADRE database is seeded once on the server and is immediately available to all users, always up to date. Authentication and access controls enable multiple users to own their own matrices and simulations, share matrices with colleagues by username, and persist simulation workspaces across sessions and devices. Deploying a new version of the application — whether a bug fix, a new COMPADRE snapshot, or a new feature — propagates to every user the moment the container restarts, with no client action required.

The API-first design (`/v1/` REST endpoints) means the simulation logic is decoupled from any particular interface, leaving the door open for future clients — a mobile application, a third-party integration, or a command-line tool — without duplicating any business logic. Security practices that are impractical in a distributed desktop application (centralised JWT [7] authentication, automated SAST with Bandit and CodeQL, dependency vulnerability scanning with pip-audit and Trivy) become straightforward in a server-hosted architecture.

The main trade-off is infrastructure dependency: the application requires an internet connection and a running server. For a public educational tool targeting university students and researchers worldwide, this is an acceptable and expected constraint — far less disruptive than requiring every user to manage a local Python environment.

3.3.1.5 Comparison and Justification

Criterion	Script	.exe	Local app	Web app
No installation required	×	✓	×	✓
Works on all platforms	~	×	~	✓
Graphical interface	×	✓	✓	✓
Live COMPADRE database	×	×	×	✓
Persistent user storage	×	×	×	✓
Multi-user collaboration	×	×	×	✓
Centralised updates	×	×	×	✓
Works without Python on client	×	✓	×	✓
99.5% availability guarantee	×	×	×	✓
Security pipeline (SAST/CVE scan)	×	×	×	✓

Table 5: Comparison of solution alternatives across key project criteria (✓ = fully satisfied, ~ = partially satisfied, × = not satisfied).

The table makes the outcome clear: the web application is the only alternative that satisfies all ten criteria simultaneously. The non-functional requirement SR-NF-Portability (SR-NF-06) explicitly mandates "Chrome, Firefox, no client installation", which eliminates the script and local desktop options outright. The non-functional requirement SR-NF-Availability (SR-NF-02) demands 99.5% monthly uptime, which requires infrastructure-level guarantees rather than per-user processes. The integration of the COMPADRE database as a shared, always-current resource is only practical when the data lives on a server accessible to all users concurrently. Collaboration features — matrix sharing, simulation persistence across devices, and future team workspaces — are architecturally impossible in any isolated, single-user deployment model.

The additional complexity of the three-tier architecture — compared to a script or a compiled binary — is justified by the breadth of capabilities it unlocks and is itself a demonstration of the full range of software engineering competencies expected of a Final Degree Project: API design, relational database modelling, containerisation, continuous integration, and automated security scanning.

3.3.2 Technology Alternatives

Every technology decision in this project was made deliberately, with concrete alternatives evaluated and rejected for specific reasons. This section documents that process for the six most consequential choices: programming language, frontend framework, backend framework, database, ORM, and migration tooling.

3.3.2.1 Programming Language — Python

The project requires three distinct capabilities from a single codebase: numerical matrix operations at the core of the simulation engine, a REST API serving JSON over HTTP, and a reactive graphical interface. The choice of programming language had to satisfy all three without forcing the use of multiple languages with separate runtimes, toolchains, and dependency trees.

R [8] was the first alternative considered given the project's roots in population ecology. R has a strong statistical computing ecosystem and its own Shiny framework for reactive web applications. However, R's production web server story is fragile — `plumber` APIs are rarely deployed outside of data science teams, and combining R with a relational database, a REST API, and a persistent user system requires bridging to tools that are not idiomatic in the language. The result would be an unusual and hard-to-maintain stack.

Node.js / TypeScript [9] is the dominant choice for REST APIs and single-page applications and would have produced an excellent backend. However, it has no equivalent to NumPy for matrix algebra. Population projection matrices require efficient n-dimensional array operations backed by optimised BLAS routines; implementing these from scratch in JavaScript is neither practical nor appropriate. A hybrid approach — TypeScript API, Python microservice for simulation — would reintroduce cross-language complexity and split the codebase.

Java / Kotlin provides robust concurrency and mature API frameworks (Spring Boot, Quarkus), but brings heavyweight tooling, verbose boilerplate, a JVM runtime requirement, and no first-class scientific computing ecosystem. The NumPy problem remains unsolved.

Python [10] resolves all three requirements simultaneously. NumPy [1] provides BLAS-backed matrix multiplication natively; FastAPI and Python Shiny handle the API and UI within the same runtime; and the entire stack can be managed with a single `requirements.txt`. Python 3.13 was chosen specifically for its interpreter performance improvements and because it matches the version available in the GitHub Actions CI runner.

3.3.2.2 Frontend Framework — Python Shiny

The use of Python for the frontend was a stakeholder constraint set by the Biology Faculty tutor. Within that constraint, three Python-native alternatives to Python Shiny were evaluated.

Streamlit [11] has an exceptionally low learning curve and is the most popular choice for rapid data prototyping. Its rendering model, however, re-runs the entire script on every interaction — a limitation that makes fine-grained state management for multi-step workflows (parameter configuration, simulation run, result inspection) require fragile session-state workarounds. Its layout system defaults to a single column, making a multi-tab application with a sidebar and a chart panel awkward to build.

Dash [12] (Plotly) is purpose-built for data dashboards and integrates naturally with Plotly charts. As the number of reactive components grows, its callback graph becomes difficult to reason about; every input-to-output relationship must be declared explicitly,

and circular dependencies or missing callbacks produce runtime errors that are hard to trace. Its layout is defined in verbose Python dict-style HTML, which produces more code than UI.

FastAPI with Jinja2 templates would give full control over the HTML and CSS, but any interactive behaviour beyond a form submission requires JavaScript — directly contradicting the stakeholder's constraint and splitting the frontend into two languages.

The reactive programming model of Python Shiny [5] — where output functions automatically re-execute when their declared inputs change — maps cleanly onto the application's interaction pattern: a set of simulation parameters driving a live population chart. The framework keeps all UI logic in Python, integrates naturally with NumPy arrays and matplotlib figures, and produces a responsive single-page experience without requiring JavaScript knowledge.

3.3.2.3 Backend Framework — FastAPI

With Python chosen as the language, three web frameworks were evaluated for the REST API: FastAPI [13], Flask [14], and Django REST Framework [15].

Flask is the most minimal of the three. It provides routing and a request/response cycle and nothing else, which means that everything the API needs — input validation, schema serialisation, OpenAPI documentation, and async support — must be assembled from separate third-party packages (marshmallow or pydantic-flask, flask-smorest, and eventually a WSGI → ASGI adapter). For an API with well-defined contracts and a strict layered architecture, this assembly cost is unnecessary overhead that Flask's flexibility does not justify.

Django REST Framework is comprehensive and production-proven. Its built-in ORM, authentication system, admin panel, pagination, and browsable API cover a wide range of use cases. However, Django's ORM is tightly coupled to Django [16] itself: using SQLAlchemy alongside Django creates a dual-ORM situation where migrations, session management, and model definitions are controlled by two separate systems. SQLAlchemy is a hard requirement here because Alembic — the chosen migration tool — depends on it directly, and the repository pattern in the service layer relies on SQLAlchemy sessions. Beyond the ORM conflict, Django's template engine, admin interface, and full-stack conventions are unused weight for a headless API whose only output is JSON.

FastAPI resolves both problems. It generates a live Swagger UI at `/docs` automatically from the route and schema definitions, with no additional packages. Input validation and response serialisation are declared as Pydantic v2 [17] model classes, which FastAPI validates and serialises natively and efficiently (Pydantic v2's Rust-backed core is substantially faster than its predecessor). The framework is async-first, built on Starlette [18], leaving the door open for non-blocking I/O without a rewrite. Its dependency injection system (`Depends`) maps directly onto the controller → service → repository architecture: a DB session, a service instance, and the authenticated user are all resolved and injected per request with a few lines of code in `api/deps.py`. Finally, FastAPI imposes no ORM preference, integrating with SQLAlchemy without conflict.

Framework	Auto OpenAPI docs	Native Pydantic	Async support	ORM flexibility	Setup overhead
FastAPI	✓	✓	✓	✓	Low
Flask	×	×	✓	✓	High
Django REST Framework	✓	×	✓	×	High

Table 6: Backend framework comparison (✓ = supported out of the box, × = not supported or requires extra packages).

3.3.2.4 Database — PostgreSQL

The data model has an unusual characteristic: several columns store structured, variable-length nested data (the population matrix components `matrix_a`, `matrix_u`, `matrix_f`; the simulation trajectory `result_history`; the list of stage labels `stage_names`; and the stochastic matrix index array `matrix_ids`). At the same time, the model relies on relational integrity — foreign keys from `simulation_runs` to `population_matrices`, unique constraints on `users.username` and `users.email`, and transactional guarantees when a simulation record is created alongside its metadata. This combination is the key constraint that drove the database decision.

MySQL / MariaDB [19] is a mature, widely-deployed relational database with comparable performance characteristics to PostgreSQL. However, it does not provide a native JSONB type. Storing variable-length nested structures would require TEXT columns with manual serialisation, losing the ability to index or query into nested fields, or a hybrid approach that offloads document-shaped data to a separate store — reintroducing the complexity that a single unified database is meant to avoid.

SQLite [20] is the natural choice for local development: zero configuration, a single file, and full SQLAlchemy compatibility. It is, however, unsuitable for the production use case. SQLite's write concurrency model uses file-level locking, which serialises all write operations and degrades severely under simultaneous requests from multiple users. It also has no native JSONB type and no production-grade connection pooling.

MongoDB [21] stores documents natively, which would handle the variable-length matrix columns elegantly without any schema overhead. The trade-off is the loss of the relational guarantees that the rest of the data model depends on: there are no enforced foreign keys, no multi-document transactions in the free tier, and no straightforward equivalent to the unique constraints on user credentials. SQLAlchemy's MongoDB support is provided through a separate ODM rather than the standard ORM interface, complicating the repository pattern and the Alembic migration workflow.

PostgreSQL [22] provides both capabilities in a single engine. Its native JSONB type stores nested structures as binary-indexed JSON, allowing efficient querying into nested fields while preserving full relational integrity around them. Version 16 introduces further performance improvements to parallel query execution and JSONB handling that are

directly relevant to the query patterns of this application — particularly list queries that filter across both relational columns and JSONB metadata fields.

Database	Native JSONB	Concurrent writes	Relational integrity	SQLAlchemy support	Production-ready
PostgreSQL	✓	✓	✓	✓	✓
MySQL	×	✓	✓	✓	✓
SQLite	×	×	✓	✓	×
MongoDB	✓	✓	×	~	✓

Table 7: Database comparison (~ = partial support via a separate ODM; ✓ = fully satisfied, × = not satisfied).

3.3.2.5 ORM — SQLAlchemy

Three approaches to database access were evaluated: Django ORM, Tortoise ORM, and raw SQL via `psycopg2`.

Django ORM [16] was excluded together with Django REST Framework. Using Django's ORM outside of a Django application requires pulling in the entire framework to initialise the ORM's application registry, which creates lifecycle and configuration conflicts with FastAPI's request handling.

Tortoise ORM [23] is an async-native ORM designed specifically for use with FastAPI and ASGI frameworks. Its API is clean and Pythonic. However, it is a younger project with a smaller ecosystem, and — critically — Alembic, the de-facto standard for SQLAlchemy migrations, does not support Tortoise ORM. Adopting Tortoise ORM would require either a different migration tool (none of which offer `--autogenerate` from model diffs as cleanly as Alembic) or managing migrations manually.

Raw SQL via psycopg2 [24] offers maximum control and the lowest query overhead. The cost is that query logic becomes tightly coupled to the database schema; any schema change requires manually updating every affected query string. More importantly for the test strategy, raw SQL queries cannot be replaced with `unittest.mock.MagicMock` objects — the entire test suite would require a live database connection, making fast, isolated unit tests impossible.

SQLAlchemy [25] uses a declarative mapping style that keeps model definitions readable and co-located with Python type annotations. Its session model integrates with FastAPI's `Depends` injection with a single generator function, scoping each session to the lifetime of a single HTTP request. ORM objects returned by repository methods can be replaced wholesale with `MagicMock` instances in unit tests, enabling the service layer to be tested in complete isolation from the database. This is the foundation of the two-speed test strategy described in the design chapter.

3.3.2.6 Database Migrations — Alembic

Three alternatives to Alembic were considered for managing schema evolution.

Django migrations are excluded together with Django ORM — they are not usable outside the Django framework.

Flyway [26] and **Liquibase** [27] are JVM-based migration tools with strong track records in enterprise Java projects. Both require a separate Java runtime installed alongside the Python application — a significant operational dependency for a Python-only stack. Neither tool has the ability to introspect SQLAlchemy model definitions and auto-generate migration scripts from model diffs, which means every migration must be written by hand in SQL.

Manual SQL scripts give the developer complete control over every DDL statement and avoid any migration framework entirely. The cost is the absence of versioning, rollback paths, drift detection, and the guarantee that the running code and the database schema are in sync. In a containerised deployment where the schema must be updated before the application starts, manual scripts require a custom orchestration layer that Alembic provides out of the box.

Alembic [28] is the direct companion library to SQLAlchemy and requires no integration work. It generates migration scripts automatically by comparing the current SQLAlchemy model definitions against the database schema (`alembic revision --autogenerate`), versions them sequentially in `alembic/versions/`, and applies them idempotently at container startup via `entrypoint.sh` (`alembic upgrade head`). Two migrations exist in the current codebase: `0001_initial_schema`, which creates the `users` and `population_matrices` tables, and `0002_simulation_runs`, which adds the `simulation_runs` table with its stochastic columns. Both run automatically on every container start, ensuring the database schema is always in sync with the deployed code without any manual intervention.

Chapter 4

System Requirements

These provide a technical representation or view of the required product, indicating what it must do to fulfill the user requirements. This involves transforming user requirements into a detailed system specification that satisfies the needs of those users and serves as a basis for subsequent system design.

As a guideline, the content of this chapter corresponds to the results of the "Requirements Analysis Process" in ISO/IEC 15288 or the "Information System Analysis (ASI)" process in the Métrica Version 3 methodology.

4.1 Functional Requirements

As a general criterion, these requirements will be expressed through three complementary views or models, each of which can be included as an independent section as indicated below.

4.1.1 System Functions

In most cases, this will be the most extensive part, and the way they are represented will depend on the methodology used. For example, they can be represented as use cases with their scenario descriptions, as user stories with their acceptance criteria, or as a hierarchical list with detailed requirements.

NOTE 1: When using use cases, avoid redundant scenario descriptions that do not add value (e.g., in simple CRUD operations, enumerating them might suffice without including detailed scenarios).

NOTE 2: When using user stories + acceptance criteria, include a story mapping first and organise the user stories according to it.

4.1.1.1 Use Case Diagram

Insert your use case diagram here as a figure (rendered externally in PlantUML, draw.io, etc. and saved as PNG/SVG under resources/).

4.1.1.2 Story Map

Authentication	Example Epic
Login	Story A
Register	Story B
Logout	

Table 8: User Story Map

4.1.1.3 User Stories

US-01 **Must** 5 pt

*As a **registered user**, I want to **log in with my email and password**, so that **access my personalized content**.*

Acceptance Criteria

- The system accepts valid email and password combinations.
- Invalid credentials show a generic error without revealing which field failed.
- A successful login redirects the user to their dashboard.

4.1.2 Domain Data Model

Graphic representation with the different data entities the system interacts with (e.g., using a class diagram).

4.1.3 User Interface

Description of the requirements and conventions of the dialogue with the user, including application navigation. If screen prototypes were created during the project, they may also be included.

4.2 Non-Functional Requirements

Depending on the project, these will be organised into one or several sections to include the rest of the requirements. Regarding non-functional requirements, security requirements are frequent, following what was taught in the third-year subject "Computer System Security".

In general, if security must be addressed transversally across the entire application, it is recommended to follow the Level 1 requirements of the OWASP ASVS. If that is not the case, and you want a minimum acceptable security level, consider the following:

- ***Do not implement your own authentication system:*** *Always use an account system managed by a third party (Google, Microsoft…). This provides more security, saves development time, and improves usability.*
- ***Do not disable copy-paste*** *in password fields to allow the use of password managers.*
- ***Do not hardcode*** *passwords, API Keys, etc., in the code; use a vault or secret store.*
- ***SCA and SAST:*** *Ensure the code passes an SCA scan (to detect vulnerable dependencies) and a SAST scan (to avoid deploying code with security errors). GitHub provides these services easily.*
- ***WAF:*** *If it is a web application deployed publicly, do not go into production without a WAF (e.g., a free CloudFlare account).*

If you do any of this, do not forget to document it properly and mention it in your presentation.

4.3 Test Plan

At a minimum, the testing strategy must be specified, identifying the different types and levels of testing, which part of the system they affect (test objects), as well as the degree of automation and tools to be used in these activities.

Explain the different layers of testing implemented:

- ***Unit tests:*** *Unit testing per module or file.*
- ***Integration tests:*** *Describe the different integration tests. Specify the database used in each case, divide integration tests by section or module, and explain each integration test.*
- ***End-to-end tests:*** *Map end-to-end tests with the different user stories; these tests validate the solution of each individual user story.*
- ***Load tests:*** *Not just a set of users performing one action. Use Gatling to create statistics and performance reports, vary the tests (e.g., incrementing the load over time) to see at which point the application breaks. Map these tests to the non-functional performance requirements.*
- ***Usability tests:*** *Design usability tests for different user profiles (biology teachers, biology students, etc.). Prepare a questionnaire, measure times, and show the results in a graphical and intuitive way.*

Chapter 5

DESIGN

This chapter documents the design decisions and structures that shape the Population Growth Simulator. It covers the system architecture, internal component design, development workflow, CI/CD pipeline, observability approach, and test design. The aim is to explain not only *what* was built but *why* each structural choice was made.

5.1 ARCHITECTURAL DESIGN

The system follows a classic three-tier architecture: a presentation layer, an application layer, and a data layer. Each tier is deployed as an independent Docker container and communicates only through well-defined interfaces, allowing each tier to be scaled, replaced, or tested in isolation.

5.1.1 HIGH-LEVEL OVERVIEW

Figure 3 shows the system boundary and its two external actors. The *User* interacts with the system through a web browser, either browsing public matrix data or running and saving simulations after authentication. The *COMPADRE Plant Matrix Database* [3] is an external data source whose matrix data is seeded into the system at startup; it has no runtime relationship with the running application.

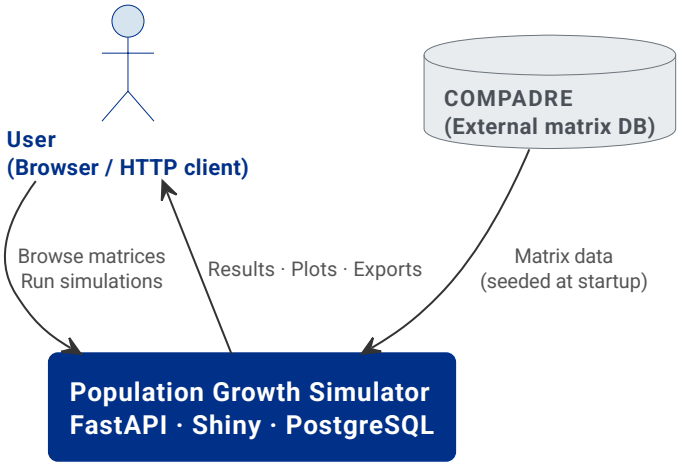

Figure 3: System context diagram. The Population Growth Simulator sits between the user's browser and the COMPADRE data source.

Figure 4 shows the three tiers and, within the application tier, the internal layering of the API service. The presentation tier (Shiny frontend, port 8080) communicates with

the application tier exclusively through HTTP REST calls. The application tier (FastAPI, port 8000) is itself structured into controllers, services, and repositories. The data tier (PostgreSQL, port 5432 inside Docker) is accessed only by the repository layer.

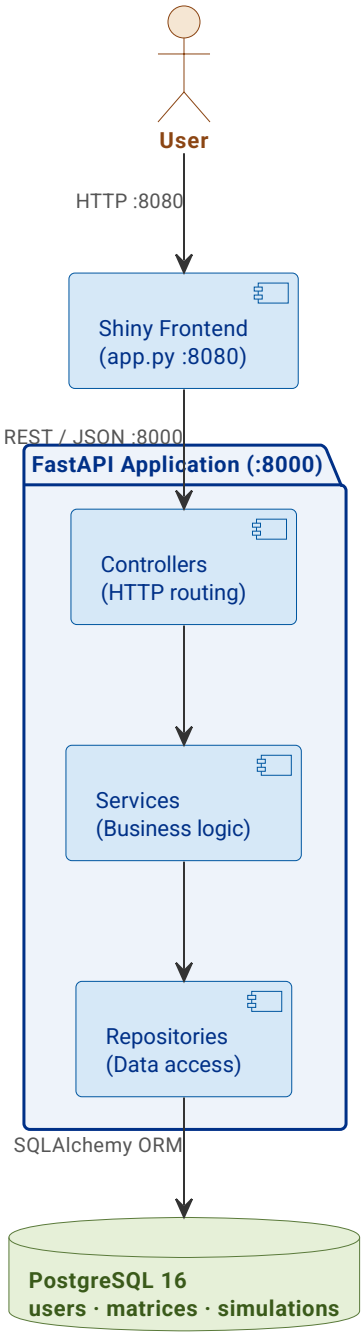

Figure 4: Building-blocks view. The three tiers and the internal controller → service → repository layering of the API.

5.1.2 Service Architecture

Within the API tier, a second layer of separation is enforced: no layer may skip another, and each has a single, well-defined responsibility.

Controllers (`api/controllers/`) handle all HTTP concerns. They parse incoming request bodies into Pydantic schema objects, delegate to the appropriate service method, and

return the resulting record with the correct HTTP status code. Controllers contain no business logic and perform no database access.

Services (`api/services/`) contain all business logic: ownership checks, immutability rules for COMPADRE matrices, cross-matrix dimension validation, and the simulation algorithm itself. Services receive Pydantic schemas as input and return domain records as output. All persistence is delegated to repositories.

Repositories (`api/repositories/`) are the only layer permitted to interact with SQLAlchemy sessions. They expose a narrow, entity-centric interface (`get_by_id`, `create`, `update`, `list`) and return raw ORM objects to the service layer. No business rule is encoded here.

Schemas and Records (`api/schemas.py`, `api/records.py`) form two distinct families of Pydantic models that cross the API boundary. Schemas (input DTOs) validate data arriving over HTTP — all field constraints and cross-field invariants live exclusively in schemas. Records (output models) represent business entities as they flow out of the service layer into HTTP responses, constructed from ORM objects via `model_validate` with `from_attributes=True`. This separation prevents accidental exposure of internal fields and makes the contract of each layer explicit.

Dependency injection (`api/deps.py`) wires the full request dependency tree using FastAPI's `Depends` mechanism: a SQLAlchemy `Session` scoped to each request, pre-built service instances receiving their repository dependencies, and the `get_current_user` guard that validates JWT tokens and resolves the authenticated user before any controller logic runs.

5.1.3 Deployment and Infrastructure

Figure 5 shows the Docker Compose deployment. The three containers share a Docker-managed bridge network; only the host-side ports listed in Table 9 are exposed externally.

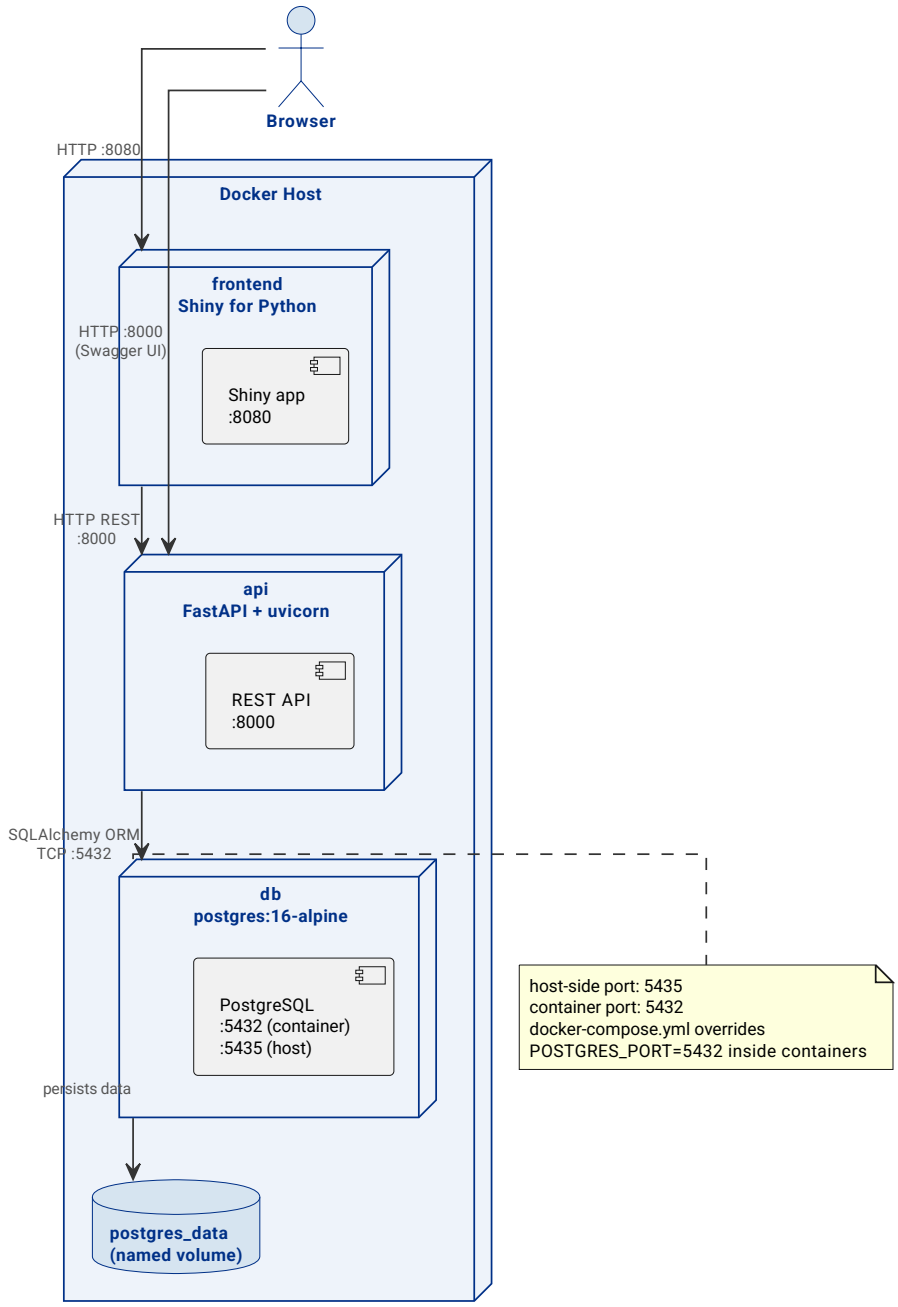

Figure 5: Deployment diagram. Three Docker containers communicate over an internal bridge network; only host-side ports are externally accessible.

Service	Image	Exposed port (host)
`db`	`postgres:16-alpine`	`5435 → 5432 (container)`
`api`	built from `Dockerfile`	`8000`
`frontend`	built from `frontend/Dockerfile`	`8080`

Table 9: Docker Compose services and exposed ports.

The `db` container uses a named volume (`postgres_data`) to persist data across container restarts. The `api` and `frontend` services declare a `depends_on` condition tied to the `db`

health check, so they only start after PostgreSQL is ready to accept connections. On startup, the `api` container runs `entrypoint.sh`, which performs three idempotent steps before launching Uvicorn: apply pending Alembic migrations (`alembic upgrade head`), seed COMPADRE data (`python db/seed_compadre.py`), and start the server. Restarting the container does not duplicate data or re-apply already-applied migrations.

A deliberate port override exists: `.env` sets `POSTGRES_PORT=5435` (the host-side port), while `docker-compose.yml` overrides `POSTGRES_PORT=5432` for the `api` and `frontend` services so that they connect to the correct container-internal port without requiring a separate environment file for Docker.

5.2 Detailed Design

5.2.1 Main System Flows

The following eight sequence diagrams document all major user journeys through the system. Each diagram captures the actors involved, the message exchange between tiers, and the key business rules enforced at each step.

User registration (Figure 6). The service layer checks that neither the chosen username nor the email is already taken before hashing the password with bcrypt and persisting the new user record.

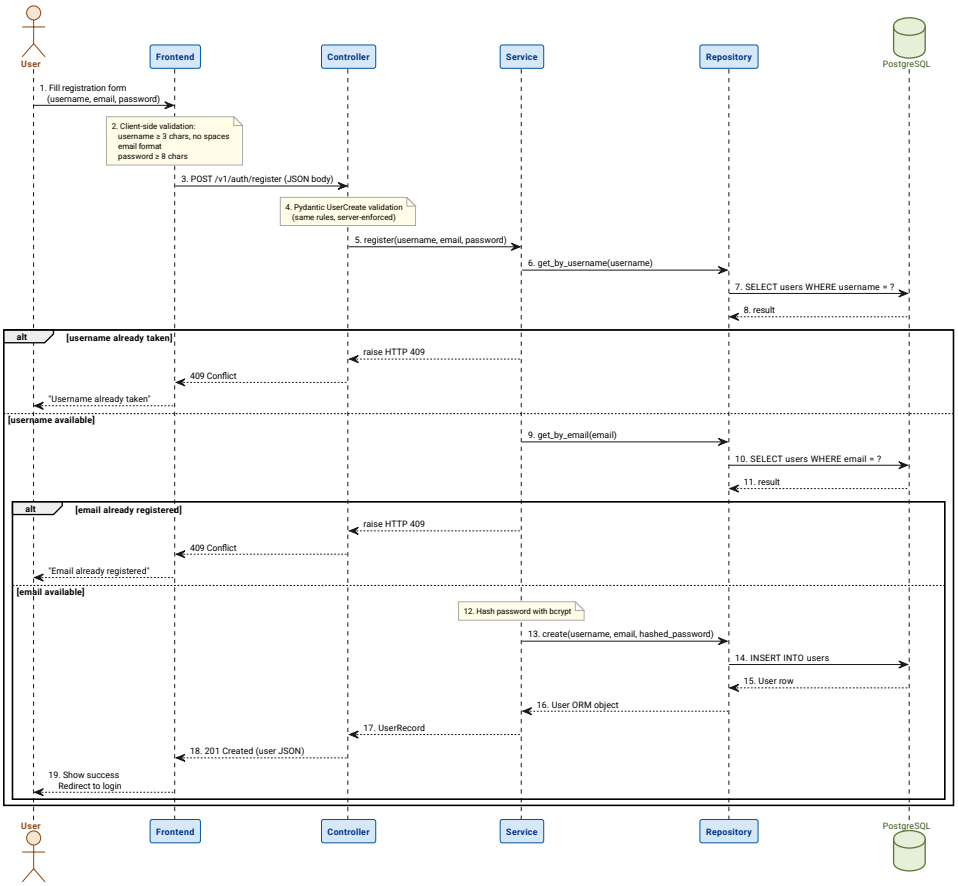

Figure 6: Sequence diagram: user registration. Uniqueness is checked before the password is hashed.

User login (Figure 7). The client submits credentials through the OAuth2 password form. The service verifies the bcrypt digest and, on success, issues a signed JWT (HS256, seven-day expiry) that the client must include as a Bearer token in subsequent authenticated requests.

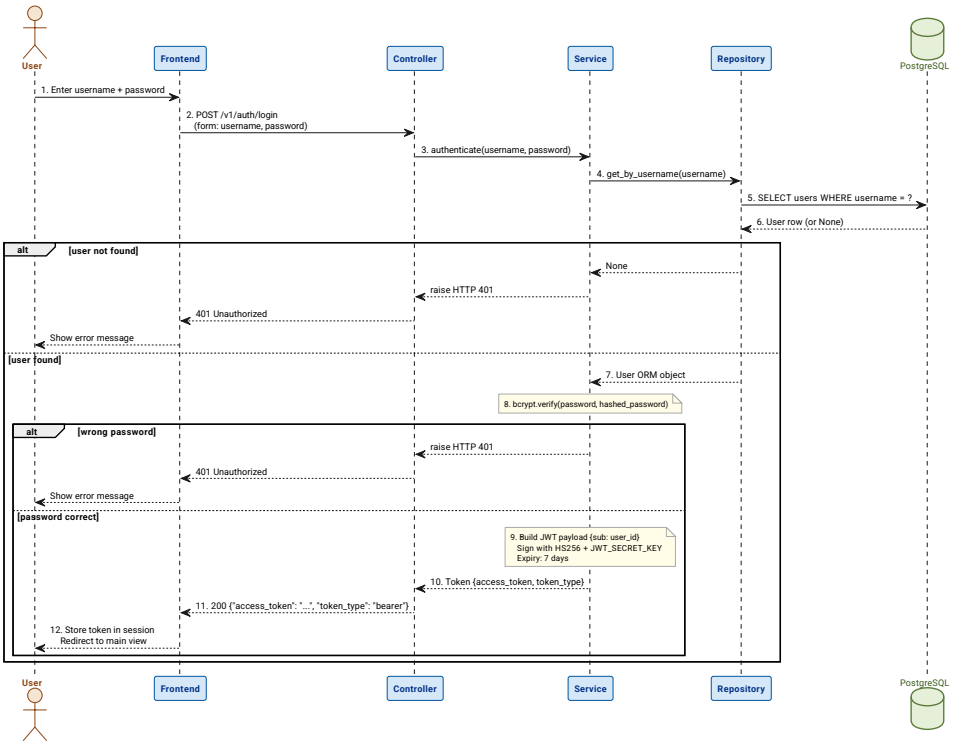

Figure 7: Sequence diagram: user login. Successful authentication produces a signed JWT.

Search and browse matrices (Figure 8). This endpoint is public — no authentication is required. The repository applies optional filters (species, kingdom, source type) and returns summary projections rather than full matrix data to keep responses compact.

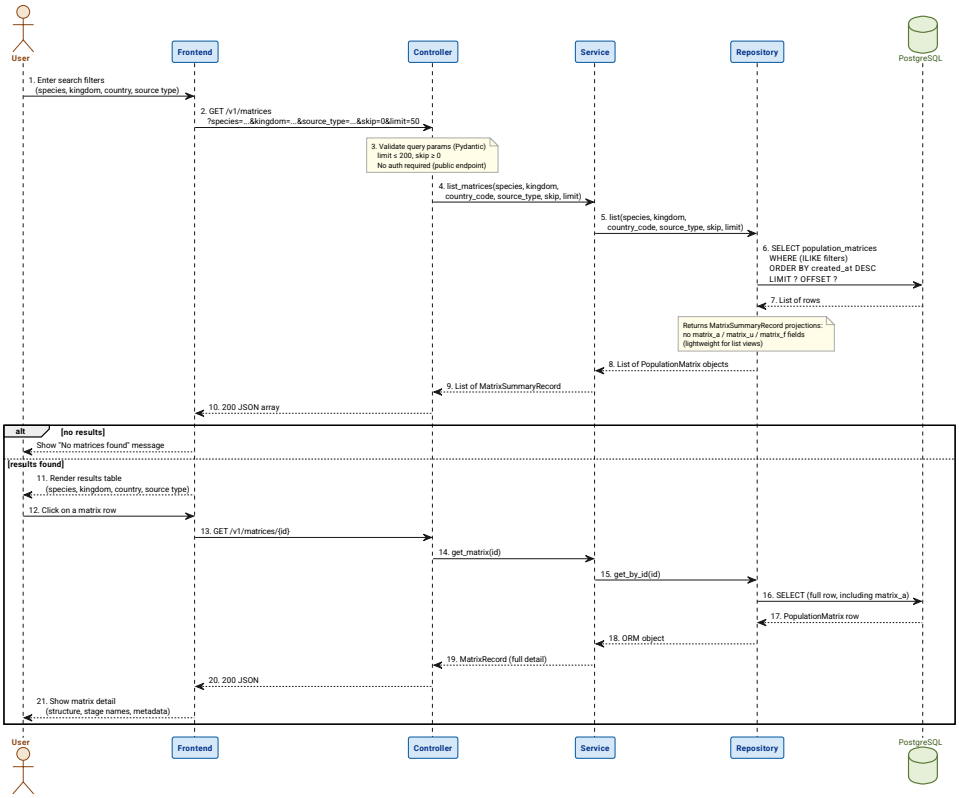

Figure 8: Sequence diagram: browse and filter matrices. The endpoint is public and returns summary projections.

Create a custom matrix (Figure 9). Authentication is required. The controller validates the request body through a Pydantic schema; the service sets the `owner_id` to the authenticated user and persists the matrix with `source_type = "custom"`.

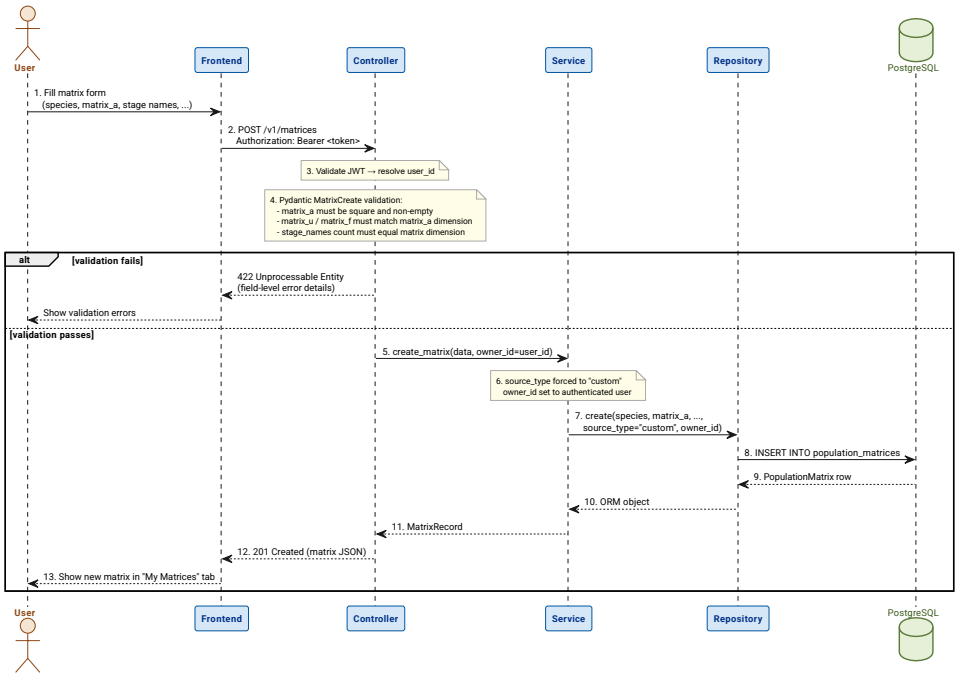

Figure 9: Sequence diagram: create a custom matrix. Ownership is set at creation time.

Save a simulation (Figure 10). The server runs the projection algorithm and stores the complete trajectory (`result_history`) alongside a snapshot of the matrix stage names.

Storing server-side guarantees reproducibility and uniform results across all client devices.

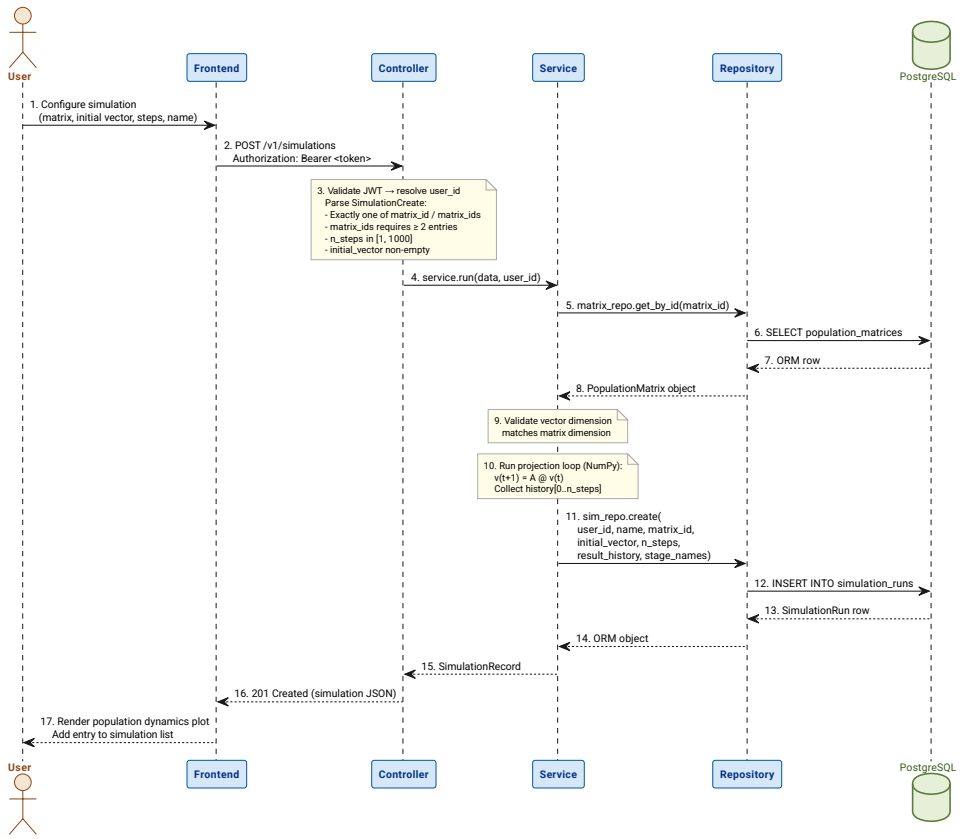

Figure 10: Sequence diagram: save a simulation. The full trajectory is computed and stored server-side.

Open a saved simulation (Figure 11). The service verifies that the requesting user owns the simulation before returning the full history. An unauthorised request receives `HTTP 403`.

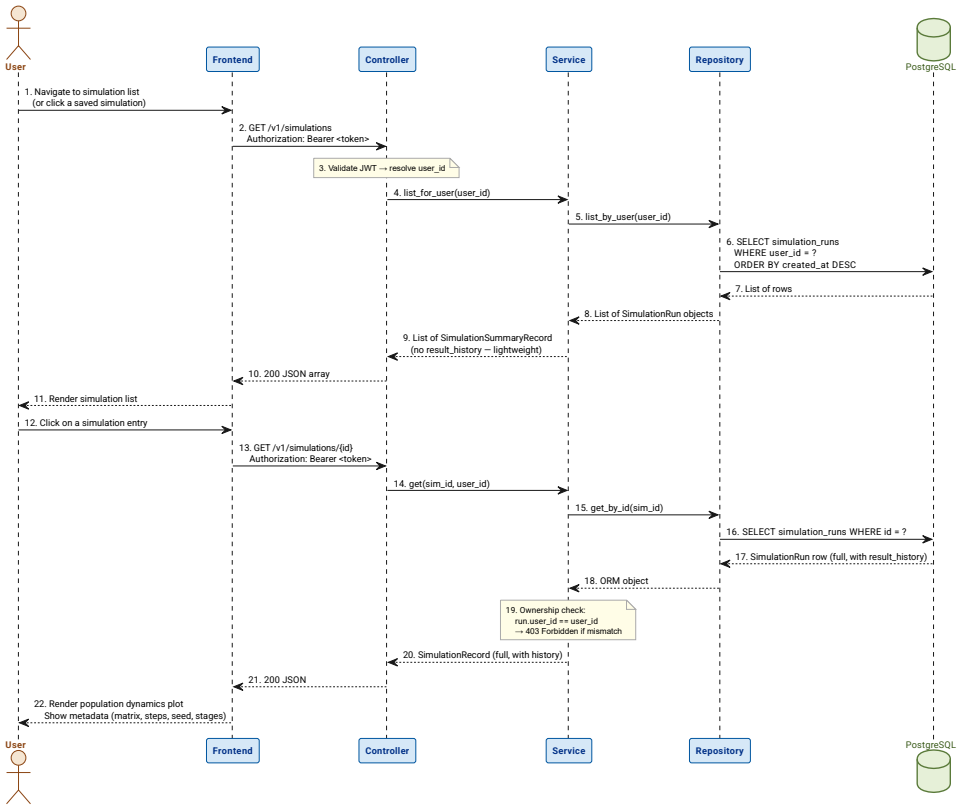

Figure 11: Sequence diagram: open a saved simulation. Ownership is checked before the trajectory is returned.

Export a simulation (Figure 12). The stored trajectory is serialised to a JSON payload and returned to the client. No recomputation occurs.

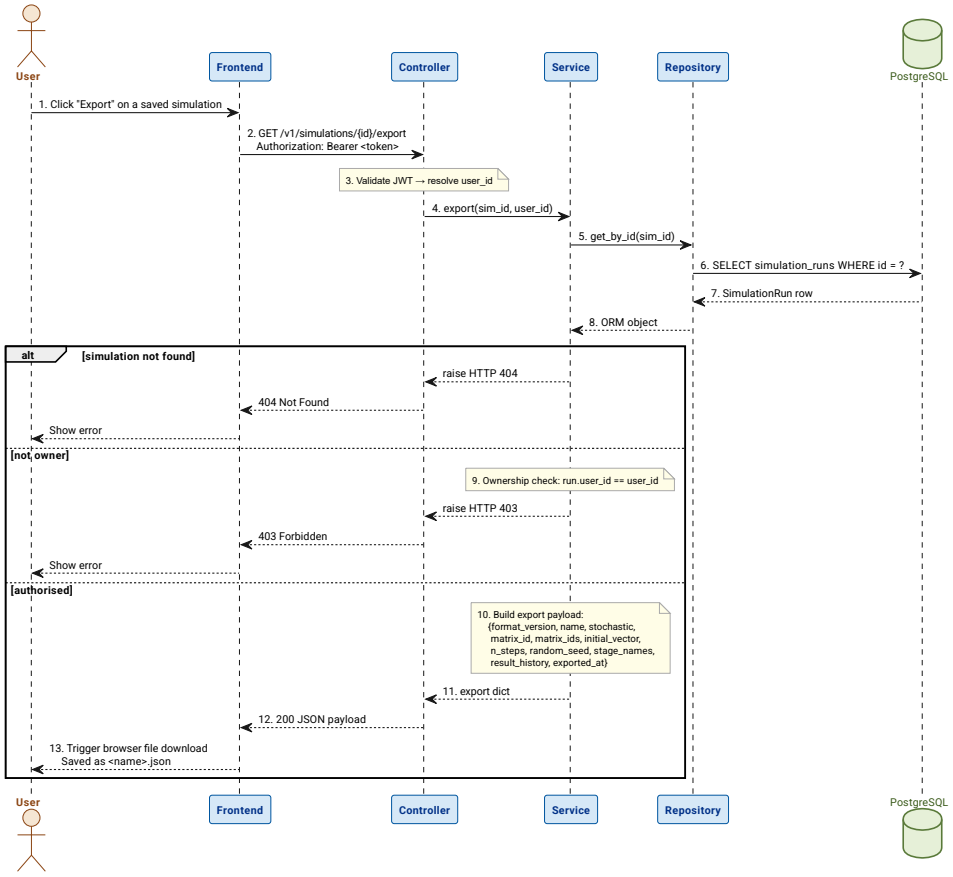

Figure 12: Sequence diagram: export a simulation as JSON. The stored result is serialised without recomputation.

Import a simulation (Figure 13). The client uploads a previously exported JSON file. The service deserialises the payload and re-persists it as a new simulation record owned by the authenticated user, without re-running the algorithm.

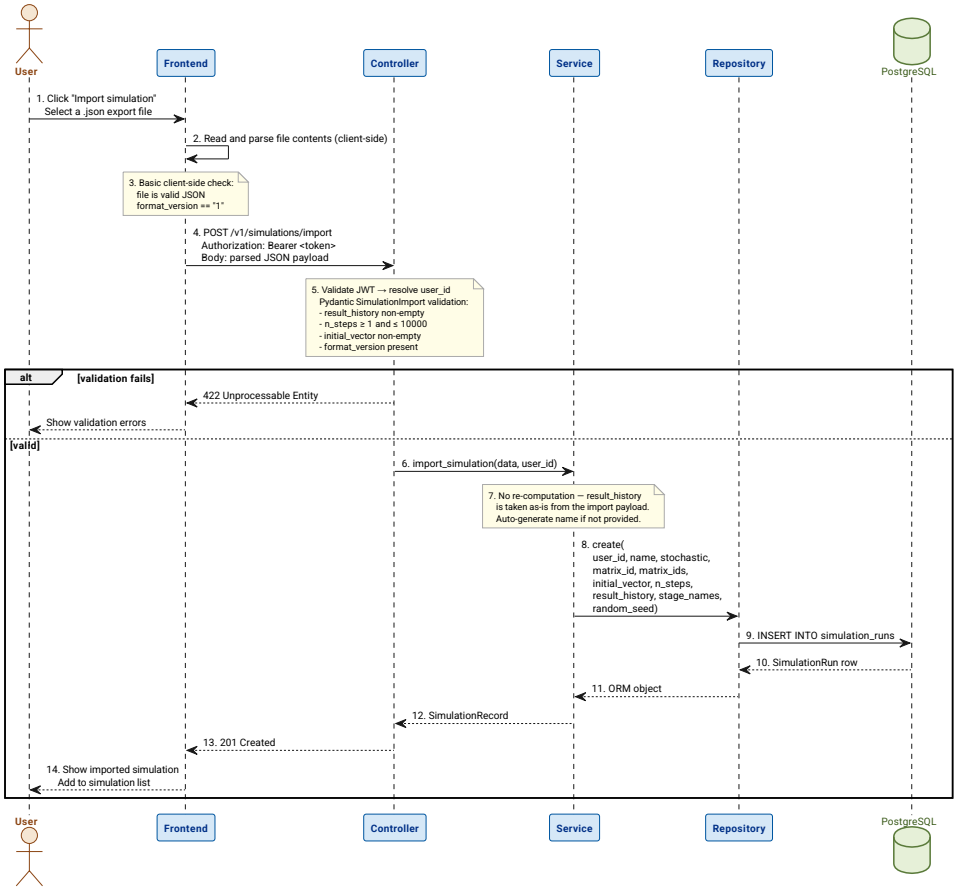

Figure 13: Sequence diagram: import a simulation from JSON. The trajectory is restored from the file without rerunning the algorithm.

5.2.2 Persistent Data Model

The database contains three tables managed by SQLAlchemy ORM models defined in `db/models.py`. Figure 14 shows the entities, their attributes, and the foreign-key relationships between them.

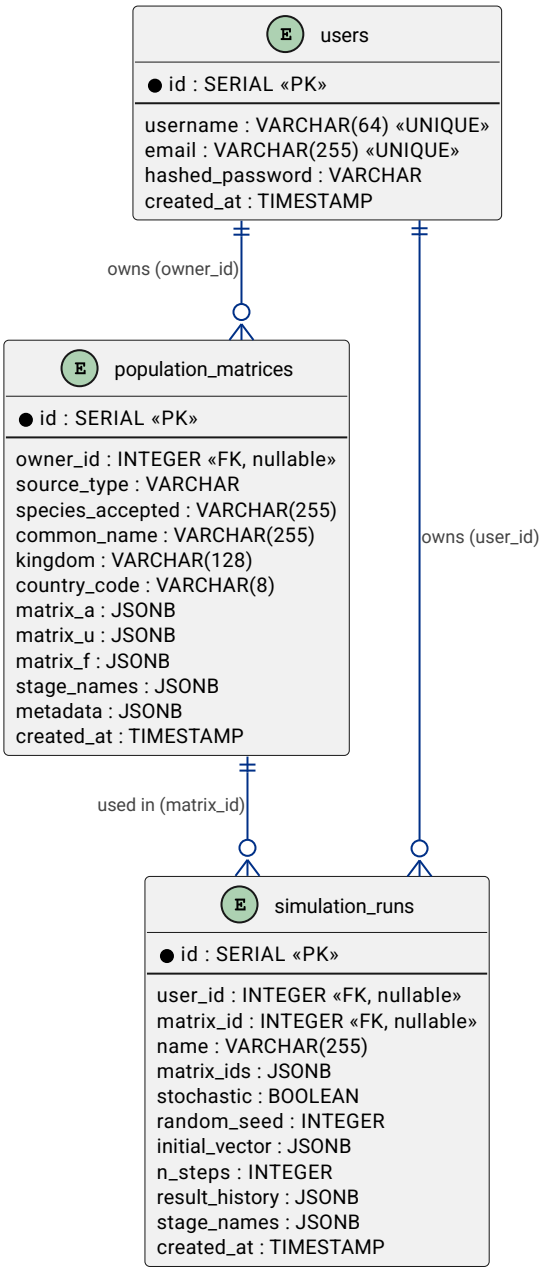

Figure 14: Entity-relationship diagram. Three tables store users, matrices, and simulation trajectories.

`users`. Stores registered user accounts. Passwords are stored as bcrypt hashes; the plaintext password exists only for the duration of the registration or login request and is never persisted. The `username` and `email` columns carry unique constraints enforced at the database level as a second line of defence after the service-layer uniqueness check.

`population_matrices`. The central reference entity. Two kinds of records coexist in this table, distinguished by `source_type`.

COMPADRE matrices (`source_type = "compadre"`) are seeded from the COMPADRE Plant Matrix Database at startup. They have `owner_id = NULL` and are read-only: any attempt to modify them through the API is rejected with `HTTP 403`. Custom matrices (`source_type = "custom"`) are created by authenticated users and can be updated by their owner.

Matrix data is stored as JSONB columns (`matrix_a`, `matrix_u`, `matrix_f`, `stage_names`, `metadata_`). The ORM column is named `metadata_` (with a trailing underscore) to avoid a collision with SQLAlchemy's internal `metadata` attribute; the API serialises it as `"metadata"` through a Pydantic `validation_alias`.

`simulation_runs`. Records a complete simulation result. The `result_history` JSONB column stores the full list of population vectors from step 0 (the initial vector) to step `n_steps`, making retrieval of individual runs self-contained without re-running the algorithm.

For deterministic runs, `matrix_id` is set and `matrix_ids` is null. For stochastic runs, `matrix_ids` is set (as a JSONB array of integers) and `matrix_id` is null. The `random_seed` column enables exact reproduction of stochastic runs. The `stage_names` column is a snapshot of the stage labels at run time, so that historical simulations remain interpretable even if the source matrix is later updated or deleted.

Schema evolution is managed by Alembic. The current migration chain is: `0001_initial_schema` (users and population_matrices) and `0002_simulation_runs` (simulation_runs table with stochastic columns). Migrations are applied automatically by `entrypoint.sh` on container startup and are idempotent.

5.2.3 Design Patterns Applied

The following design patterns are applied throughout the codebase. Each is documented with its intent and the concrete classes that fulfil each role.

Repository Pattern. Intent: decouple persistence from business logic, making services independently testable. Roles: abstract interface → implicit Python duck typing; concrete repositories → `MatrixRepository` (`api/repositories/matrix_repository.py`), `SimulationRepository`, and `UserRepository`. The service layer never accesses SQLAlchemy sessions directly; it always goes through a repository. In the test suite, repositories are replaced with `unittest.mock.MagicMock` objects so that service logic can be verified without a database.

Dependency Injection. Intent: provide service and repository instances to controllers without coupling them to instantiation logic. Roles: injector → FastAPI's `Depends` mechanism; provider → `api/deps.py`; consumers → all controller route functions. Each request receives a fresh DB session scoped to its lifetime; services and repositories are constructed within `deps.py` and injected into the controllers that need them.

DTO / Record Split. Intent: maintain a strict boundary between input validation and output serialisation. Roles: input DTO → Pydantic Schema classes in `api/schemas.py`; output model → Record classes in `api/records.py`. This separation prevents the accidental exposure of internal ORM fields in responses and makes the API surface explicit and auditable.

Immutability Guard. Intent: protect the integrity of the COMPADRE reference dataset regardless of the requesting user's identity. Roles: guard location → `MatrixService.update_matrix` in `api/services/matrix_service.py`; trigger → `source_type`

`== "compadre"`; response ⟶ `HTTPException(status_code=403)`. The rule is enforced entirely in the service layer with no reliance on a database constraint, making it fully testable with mocked repositories.

5.3 Development Workflow and Tooling

5.3.1 Repository and Version Control

The project is hosted on GitHub. Figure 15 shows the branching model used throughout development.

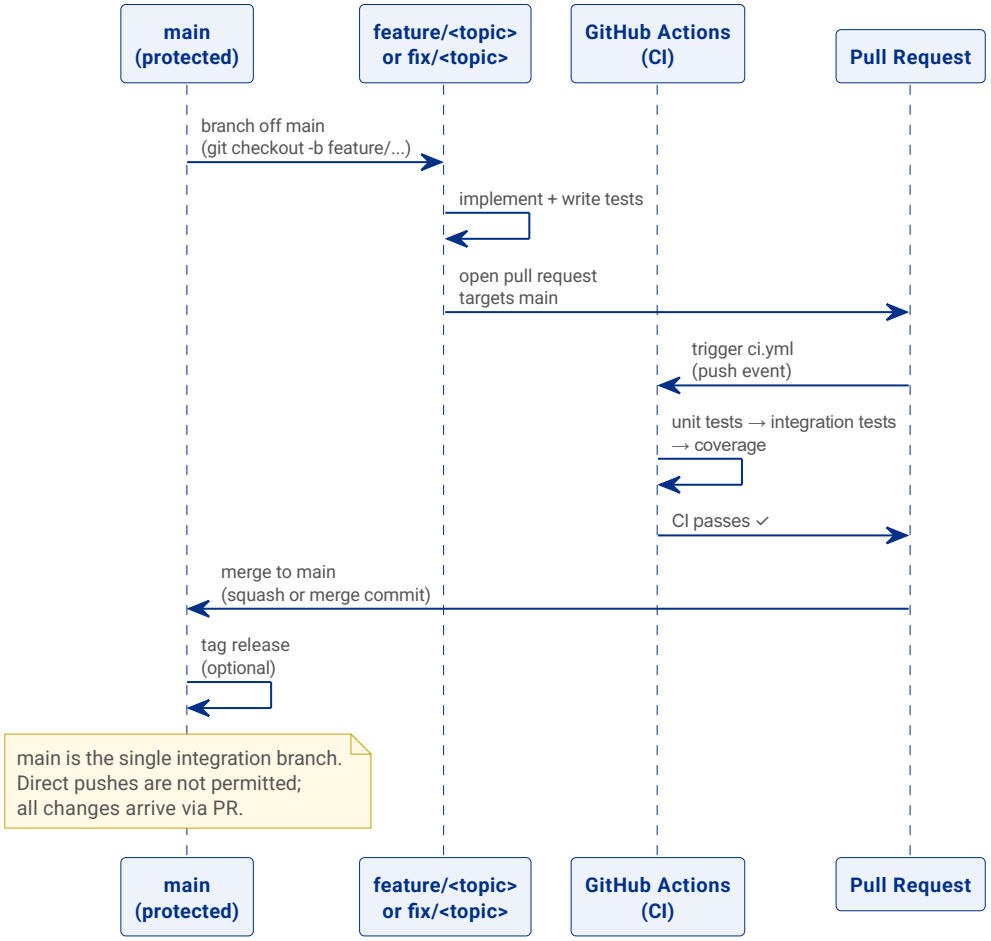

Figure 15: Branch strategy. All changes arrive on `main` through a pull request; direct pushes are not permitted.

The `main` branch is the single integration branch and is kept in a deployable state at all times. All new work is developed on short-lived feature or fix branches named `feature/<topic>` or `fix/<topic>`. A pull request targeting `main` is required before any branch can be merged; the CI workflow (Section sec:ci) must pass before the merge is allowed. This guarantees that `main` never contains code that fails the test suite.

5.3.2 Team Workflow

This is a single-developer project. The development cycle follows a lightweight agile-inspired loop: identify a task or defect, create a feature branch from `main`, implement the change together with its tests, open a pull request, wait for CI to pass, and merge. Commit messages follow the imperative-mood convention (e.g. `Add stochastic simulation endpoint`, `Fix COMPADRE immutability check`), keeping the git log readable as a chronological record of intent.

5.4 Continuous Integration and Deployment (CI/CD)

5.4.1 Pipeline Overview

Figure 16 shows the two GitHub Actions workflows that together form the automated pipeline. Table 10 summarises their triggers and purposes.

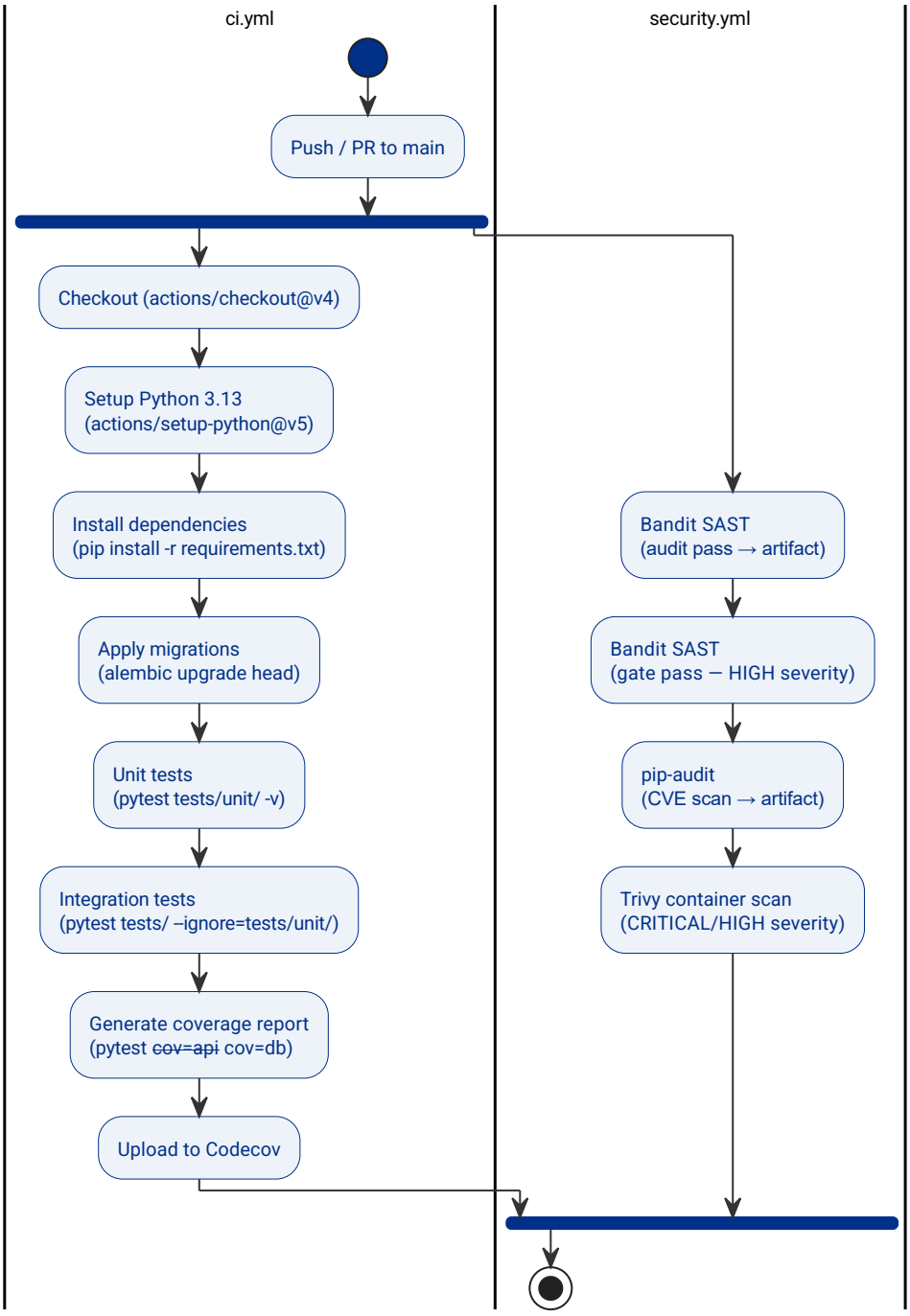

Figure 16: CI/CD pipeline. Two parallel GitHub Actions workflows handle correctness and security concerns independently.

File	Trigger	Purpose
`ci.yml`	Push / PR to `main`	Correctness — tests and coverage
`security.yml`	Push / PR to `main`; weekly (Monday)	Security — SAST, dependency audit, container scan

Table 10: GitHub Actions workflow files.

The two workflows are kept separate deliberately. A security job failure (for example, a newly disclosed CVE in a transitive dependency) does not block a feature merge, because the fix may require an upstream release that is outside the developer's immediate control. Conversely, the weekly schedule of the security workflow surfaces vulnerabilities in base images and dependencies that no code commit could have detected.

5.4.2 Continuous Integration

The CI workflow (`ci.yml`) runs on every push and pull request targeting `main`. It provisions a PostgreSQL 16 service container within the CI job so that integration tests can run against a real database without any external infrastructure.

The workflow executes the following steps in order:

1. **Checkout** — `actions/checkout@v4` fetches the repository at the triggering commit.
2. **Setup Python 3.13** — `actions/setup-python@v5` installs Python and enables pip dependency caching.
3. **Install dependencies** — installs `requirements.txt` plus the test packages (`pytest`, `pytest-cov`).
4. **Apply migrations** — `python -m alembic upgrade head` brings the CI database schema to the current version.
5. **Unit tests** — `pytest tests/unit/ -v` runs the fast, database-free unit tests first. Failures here abort the pipeline before slower integration tests are attempted.
6. **Integration tests** — `pytest tests/ --ignore=tests/unit/ -v` runs the full HTTP integration suite against the provisioned PostgreSQL instance.
7. **Generate coverage report** — a combined `--cov=api --cov=db` run produces both a terminal summary and an XML report.
8. **Upload to Codecov** — the XML report is sent to Codecov for trend tracking. `fail_ci_if_error: false` prevents a Codecov service outage from blocking the pipeline.

Table 11 lists the environment variables configured for the CI job. The `JWT_SECRET_KEY` value is a non-production placeholder used only within the isolated CI environment.

Variable	Value in CI
`POSTGRES_USER`	`postgres`
`POSTGRES_PASSWORD`	`postgres`
`POSTGRES_DB`	`matrix_db`
`POSTGRES_HOST`	`localhost`
`POSTGRES_PORT`	`5432`
`JWT_SECRET_KEY`	`ci-test-secret-key-not-used-in-production`

Table 11: Environment variables configured for the CI job.

5.4.3 Continuous Deployment

No automated deployment pipeline exists in the current implementation. Deployment is performed manually by running `docker compose up -d --build` on the target host after pulling the latest `main` branch. Automating this step — for example, building a tagged Docker image in CI and deploying it to a cloud host on merge to `main` — is identified as future work.

5.5 Monitoring Design

5.5.1 Observability Strategy

A lightweight observability approach was chosen, appropriate to the educational and research focus of the application. Two mechanisms are in place.

Uvicorn emits structured access logs by default for every HTTP request, capturing the method, path, response status code, and processing time. These logs are written to standard output and collected by Docker, making them visible via `docker compose logs -f api` during local development and available to any log aggregation tool in a cloud deployment.

Test coverage is tracked via Codecov on every push to `main`. The CI pipeline uploads an XML coverage report for `api/` and `db/`, giving a continuous, visible measure of how much of the codebase is exercised by the test suite and surfacing coverage regressions in pull request reviews.

5.5.2 Monitoring Architecture

The `GET /health` endpoint returns a static `200 OK` response with no authentication required. It serves as a liveness probe for Docker Compose health checks: the `api` container is only considered ready after this endpoint responds successfully, which gates the `frontend` container from starting before the API is available. The same endpoint can be polled by an external load balancer or uptime monitor in a production deployment.

The `db` container is configured with a PostgreSQL-specific health check (`pg_isready`) that gates the `api` and `frontend` containers through Docker Compose `depends_on` conditions. This prevents the application from starting against a database that has not yet finished initialising.

5.5.3 Dashboard and Alert Design

Prometheus metrics collection, Grafana dashboards, and alert rules are not implemented in the current release. This is a deliberate scope decision: adding a Prometheus exporter (e.g. `prometheus-fastapi-instrumentator`), a Prometheus scrape configuration, and Grafana dashboard definitions would meaningfully increase the operational surface area without adding educational or research value in this phase. Integrating full observability infrastructure is identified as future work in Section sec:future-work.

5.6 Test Design

The test suite is organised around a two-speed philosophy. Fast unit tests with no external dependencies provide immediate feedback during development; thorough integration tests against a real PostgreSQL instance validate the full HTTP contract, SQL correctness, and migration state.

Unit tests (`tests/unit/`) use `unittest.mock.MagicMock` to substitute repository implementations, allowing service business logic and Pydantic schema validators to be exercised in complete isolation from the database. The suite covers `MatrixService`, `SimulationService`, `AuthService`, and the schema validation edge cases defined in `api/schemas.py`. Unit tests complete in seconds and are run first in CI; a failure here aborts the pipeline before integration tests are attempted.

Integration tests (`tests/`) use FastAPI's `TestClient` against a dedicated `matrix_db_test` database. Shared pytest fixtures (`alice`, `bob`, `compadre_matrix_id`, `alice_matrix`) create and tear down test data per session, keeping tests isolated from one another. This layer validates full request-to-database-to-response round-trips, including authentication flows, ownership rules, and the simulation computation path.

End-to-end tests (`tests/e2e/`) use Playwright for Python to drive the Shiny frontend in a real browser. Two run modes are supported: `RUN_MODE=mock` serves canned fixture data through a lightweight mock API server (no database required); `RUN_MODE=real` exercises the full stack. E2E tests are currently run manually and are not part of the CI pipeline.

The test plan (what is tested and why) is described in Section 4.3. Section 6.4 will present the implementation results and coverage numbers.

Chapter 6

Implementation

6.1 Application Structure

Describe here how your application is organised so that the reader understands its different modules and deployed parts. This section is "free-form," and you should consult with your tutor. Some guidelines to consider are:

- *You can base this on the "Building Block View" and "Deployment View" sections of the ARC42 templates mentioned in the "Software Architecture" subject: arc42.org/overview#building-block-view*

- *Please, do not repeat elements (this applies to the entire documentation). If you have already discussed something, simply reference the section where you did so (using cross-references). You can also insert links to external data sources.*
- *Do not copy and paste the entire code into this document; submit it as an attached file (unless there are specific parts that you must highlight for a particular reason).*

Package diagram / module tree

6.2 CI/CD Pipeline Implementation

6.2.1 Repository Configuration

6.2.2 Branch Strategy and Protection Rules

6.2.3 Implemented Pipeline Stages

Screenshots of running pipeline

6.2.4 Secret and Environment Variable Management

6.3 Monitoring Implementation

6.3.1 Application Instrumentation

6.3.2 Prometheus Configuration

6.3.3 Grafana Dashboards

Screenshots of real dashboards

6.3.4 Configured Alerts

If applicable

6.4 Implementation of Tests

Details of the tests designed according to the test plan will be included here. For example, it may include scripts for manual test execution (if performed), references to automatic execution scripts (e.g., pytest), as well as test execution results that are of interest.

Note: It is highly recommended to include a summary table of the test results, showing the percentage of passed/failed tests and code coverage if applicable.

Chapter 7

Manuals

In the event that manuals are included, a clear distinction must be made regarding their target audience, especially between technical manuals (e.g., installation, operation) and user manuals.

Typically, in a manual for an end user, it is more important to show them how they can perform their processes using the application rather than providing a detailed guide of every single screen. Conversely, in an installation manual, it can be crucial to provide very detailed information to reproduce the installation environment.

7.1 User Manual

Focus on use cases and the value provided to the end user. Use screenshots to illustrate the main workflows.

7.2 Installation and Configuration Manual

Detail the prerequisites (OS, versions of languages/frameworks), the deployment steps, and how to verify that the system is running correctly.

7.3 Operations and Monitoring Manual

Chapter 8

Conclusions and Future Work

8.1 Conclusions

System conclusions: What has been developed, whether the results are within expectations, whether expectations have been met, and the justification for having chosen the best options for each aspect of the system, etc.

It is recommended to relate this section back to the objectives defined in Chapter 1 to demonstrate that they have been achieved.

8.2 Future Work

Any expansion work or future developments contemplated for the system must be described here, mentioning what they consist of, how they would expand the system, what advantages they provide, and why they were not included in the current designed system, among other aspects.

Chapter 9

Appendices

9.1 Risk Management Plan

This section is COMPULSORY. You must follow the instructions provided by the 4th-year subject "Project Management and Planning".

9.2 Bibliographic References

Please do not confuse a reference list with a general bibliography.

- ***References:*** *Includes only sources directly cited in your text. Use* `@key` *in Typst to cite. This provides the necessary information for the reader to locate the specific sources used to support your arguments.*
- ***General Bibliography:*** *A broader list of sources consulted or relevant to your research, whether cited or not. All sources that appear in* `sources.bib` *and are cited in the text will appear in the bibliography section.*

The reference to this template must be kept as follows:

[1] J. M. Redondo, "Documentos-modelo para Trabajos de Fin de Grado/Master de la Escuela de Ingeniería Informática de Oviedo," 17 6 2019. [Online]. Available: https://www.researchgate.net/publication/327882831_Plantilla_de_Proyectos_de_Fin_de_Carrera_de_la_Escuela_de_Informatica_de_Oviedo

9.3 Content Delivered in Appendices

9.3.1 Description of Content

University platforms usually have a file size limit (40–90 MB). If your attachment is larger, create a `README.TXT` and provide a link to the university's OneDrive or a GitHub repository.

The directory structure of the attached file should be organised as follows:

Directory	Content
`./`	Contains a `README.TXT` explaining this structure.
`./<project_name>`	Development directory structure (see Table 13).
`./installation`	Files for installation or a document explaining where to download 3rd party software and corresponding versions.
`./documentation`	PDF documentation (this document and others).
`./exploitation`	Database scripts and 3rd party software requirements for execution (e.g., Web Server).

Table 12: General structure of the attached annex file.

9.3.2 Recommended "Development" Directory Structure

Directory	Content
`./`	IDE project files.
`./src`	Source code files.
`./src/sql`	SQL scripts for database construction and initial data.
`./lib`	External libraries (.jar, .dll, …) required for compilation and distribution.
`./test`	Base directory for all automated testing files.
`./doc`	Files generated by automatic tools like Javadoc.

Table 13: Recommendation for the "development" folder structure.

BIBLIOGRAPHY

[1] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[2] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[3] R. Salguero-Gómez *et al.*, "The COMPADRE Plant Matrix Database: an open online repository for plant demography," *Journal of Ecology*, vol. 103, no. 1, pp. 202–218, 2015, doi: 10.1111/1365-2745.12334.

[4] PyInstaller Development Team, "PyInstaller." [Online]. Available: https://pyinstaller.org/

[5] Posit PBC, "Shiny for Python." [Online]. Available: https://shiny.posit.co/py/

[6] Docker, Inc., "Docker." [Online]. Available: https://www.docker.com/

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7519

[8] R Core Team, "R: A Language and Environment for Statistical Computing." Vienna, Austria, 2024. [Online]. Available: https://www.r-project.org/

[9] OpenJS Foundation, "Node.js." [Online]. Available: https://nodejs.org/

[10] Python Software Foundation, "Python Programming Language." [Online]. Available: https://www.python.org/

[11] Streamlit Inc., "Streamlit." [Online]. Available: https://streamlit.io/

[12] Plotly Technologies Inc., "Dash." [Online]. Available: https://dash.plotly.com/

[13] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com/

[14] Pallets Projects, "Flask." [Online]. Available: https://flask.palletsprojects.com/

[15] T. Christie, "Django REST Framework." [Online]. Available: https://www.django-rest-framework.org/

[16] Django Software Foundation, "Django." [Online]. Available: https://www.djangoproject.com/

[17] S. Colvin, "Pydantic." [Online]. Available: https://docs.pydantic.dev/

[18] T. Christie, "Starlette." [Online]. Available: https://www.starlette.io/

[19] Oracle Corporation, "MySQL." [Online]. Available: https://www.mysql.com/

[20] D. R. Hipp, "SQLite." [Online]. Available: https://www.sqlite.org/

[21] MongoDB, Inc., "MongoDB." [Online]. Available: https://www.mongodb.com/

[22] The PostgreSQL Global Development Group, "PostgreSQL." [Online]. Available: https://www.postgresql.org/

[23] Tortoise ORM Contributors, “Tortoise ORM.” [Online]. Available: https://tortoise.github.io/

[24] Psycopg Team, “Psycopg – PostgreSQL database adapter for Python.” [Online]. Available: https://www.psycopg.org/

[25] M. Bayer, “SQLAlchemy.” [Online]. Available: https://www.sqlalchemy.org/

[26] Redgate Software, “Flyway.” [Online]. Available: https://flywaydb.org/

[27] Liquibase Inc., “Liquibase.” [Online]. Available: https://www.liquibase.org/

[28] M. Bayer, “Alembic.” [Online]. Available: https://alembic.sqlalchemy.org/